BPF Syntax Basics

Capture filters are built from:

- **Qualifiers**: Specify what the primitive applies to.
    - **Direction Qualifiers**: src (source), dst (destination). Default is both.
    - **Type Qualifiers**: host (IP or hostname), net (network), port (TCP/UDP port), portrange (port range).
    - **Protocol Qualifiers**: ether (Ethernet), ip (IPv4), ip6 (IPv6), arp, tcp, udp, etc.
- **Primitives**: The core matching elements, like an IP address or port number.
- **Operators**: Combine with and (or &&), or (or ||), not (or !).
- **Parentheses**: Use (and) for grouping complex filters.

General format: [dir] [type] [proto] primitive [operator other stuff]

If no qualifier, it assumes host for IPs/hostnames, port for numbers.

Common Primitives and Qualifiers with Examples

Let's use a table for clarity. Each example includes what it does and how to test it on your PC. Replace IPs/ports with your own (e.g., use ipconfig or ifconfig to find your local IP).

Category	Filter Example	Description	How to Test on Your PC
Basic IP/Host	host 192.168.1.1	Captures traffic to/from this IP (IPv4 or IPv6).	Ping 192.168.1.1 (your router?) and capture. Expect only ping packets.
	host www.google.com	Captures to/from this hostname (resolves to IP).	Open google.com in browser; only Google traffic shows.
	src host 192.168.1.100	Only from this source IP.	Use your PC's IP; browse a site—only outgoing packets captured.
	dst host 8.8.8.8	Only to this destination IP (Google DNS).	Run ping 8.8.8.8; only those packets appear.
Networks	net 192.168.1.0/24	Traffic to/from this subnet (CIDR notation).	Capture local network traffic; browse internally.

	net 192.168.0.0 mask 255.255.255.0	Same as above, using mask (older style).	Same as above.
	src net 10.0.0.0/8	From private 10.x network.	If on VPN, test internal pings.
Ports	port 80	TCP/UDP port 80 (HTTP).	Browse a non-HTTPS site; only port 80 traffic.
	tcp port 443	Only TCP port 443 (HTTPS).	Open https://example.com; capture secure traffic.
	udp port 53	UDP port 53 (DNS).	Run nslookup google.com; only DNS queries/responses.
	portrange 5000-6000	Ports 5000 to 6000 (TCP/UDP).	If running a server on those ports, connect to it. (Requires libpcap 0.9+.)
	src port 1234	From source port 1234.	Rare, but test with custom apps.
Protocols	tcp	All TCP traffic.	Browse web; no UDP like DNS unless combined.
	udp	All UDP.	Capture while streaming video (often UDP).
	ip	Only IPv4.	Excludes IPv6; test on dual-stack network.
	ip6	Only IPv6.	Ping ipv6.google.com.
	arp	ARP protocol (address resolution).	Ping a new device on LAN; capture ARP requests.
	ether proto 0x0800	Ethernet type for IPv4 (hex).	Same as ip, but at L2.
MAC Addresses	ether src 00:11:22:33:44:55	From this source MAC.	Use your device's MAC (check with ip link on Linux).

	ether dst ff:ff:ff:ff:ff:ff	To broadcast MAC.	Capture broadcasts like DHCP.
Multicast/Broadcast	not broadcast	Exclude broadcasts.	Capture clean unicast traffic.
	not multicast	Exclude multicasts.	Useful on Wi-Fi networks.

Logical Operators and Combining Filters

Use and, or, not to build complex rules. Parentheses are key for order.

Examples:

- host 192.168.1.1 and tcp port 80: Traffic to/from 192.168.1.1 on TCP port 80. Test: Serve HTTP on that IP.
- port 80 or port 443: HTTP or HTTPS. Test: Browse mixed sites.
- not (port 53 or arp): Everything except DNS and ARP. Test: Browse; no resolution packets.
- host www.example.com and not (port 80 or port 25): Non-HTTP/SMTP on example.com. Test: SSH to a server if possible.
- tcp port 80 and not host 192.168.1.1: HTTP but not from/to that IP. Test: Exclude local traffic.
- (ip src 192.168.1.0/24 or ip6 src fe80::/10) and port 443: Secure traffic from local IPv4 or link-local IPv6. Test: On dual-stack setup.

Exercise 1: Basic HTTP Traffic

port 80

Exercise 2: Traffic to a Specific IP

host 192.168.1.1 (replace with your actual router IP)

Exercise 3: DNS Queries Only

udp port 53

Exercise 4: Exclude Local Broadcasts

not broadcast

Exercise 5: HTTPS from a Specific Source

src host 192.168.1.100 and tcp port 443

Exercise 8: Non-HTTP/HTTPS Web Traffic

tcp and not (port 80 or port 443)

Exercise 9: IPv6 Only to a Host

ip6 host ipv6.google.com

1. HTTP or HTTPS Traffic

- **Filter**: tcp port 80 or tcp port 443

2. Traffic to/from a Host Excluding DNS

- **Filter**: host 8.8.8.8 and not port 53

3. TCP or UDP on Specific Ports

- **Filter**: (tcp port 80 or udp port 123) and host 192.168.1.1

4. Non-Local Traffic on High Ports

- **Filter**: portrange 1024-65535 and not net 192.168.0.0/16

5. SYN or FIN Packets with ACK

- **Filter**: tcp[tcpflags] & (tcp-syn|tcp-fin) != 0 and tcp[tcpflags] & tcp-ack != 0

6. IPv4 or IPv6 Broadcasts

- **Filter**: (ip broadcast or ip6 multicast) and ether src 00:11:22:33:44:55

7. HTTP Requests Excluding Certain Hosts

- **Filter**: tcp port 80 and not (host 192.168.1.100 or host 10.0.0.1)

8. Complex: External UDP Not DNS or NTP

- **Filter**: udp and not (port 53 or port 123) and not src net 192.168.1.0/24

DISPLAY FILTERS

Display Filter Syntax Basics

Filters are built from:

- **Fields**: Protocol-specific elements, like ip.src (source IP) or http.request.method.
- **Operators**: Compare values (e.g., ==, >, contains).
- **Values**: What you're matching (IPs, strings, numbers).
- **Logical Connectors**: and (&&), or (||), not (!), in for sets.
- **Parentheses**: For grouping complex expressions.

General format: field operator value [logical other stuff]

- Fields are case-sensitive and use dots (e.g., tcp.port).
- Strings use quotes: "GET".
- No spaces around operators.
- Wireshark knows thousands of fields—type partway and hit Tab for suggestions.

3. Operators in Detail

Use these to compare fields:

Operator	Description	Example
`==` or `eq`	Equals	`ip.src == 192.168.1.1`
`!=` or `ne`	Not equals	`tcp.port != 80`
`>` or `gt`, `>=` or `ge`	Greater than/equal	`frame.len > 1000` (packets >1000 bytes)
`<` or `lt`, `<=` or `le`	Less than/equal	`http.time < 0.5` (responses <0.5s)
`contains`	String contains substring	`http contains "login"`
`matches` or `~`	Regex match	`http.user_agent matches ".*Chrome.*"`
`in`	Value in set	`tcp.port in {80 443}`

Logical: Combine with `and`, `or`, `not`. E.g., `(http or dns) and ip.src == 10.0.0.1`.

A. Basic Protocol Filters

These show all packets of a protocol.

Filter	What It Does	How to Test
`http`	All HTTP packets	Browse a site; only HTTP shows (no TCP handshakes).
`dns`	DNS queries/responses	Run `nslookup`; see domain lookups.
`tcp`	All TCP	Capture browsing; hides UDP/ARP.
`udp`	All UDP	Stream video; shows RTP/UDP packets.
`arp`	ARP only	Ping new LAN device; see resolutions.

B. IP and Address Filters

Filter by source/destination.

Filter	What It Does	How to Test
`ip.src == 192.168.1.100`	From your PC's IP	Browse; only outgoing. Replace with your IP.
`ip.dst == 8.8.8.8`	To Google DNS	Ping 8.8.8.8; only those.
`ip.addr == 10.0.0.0/24`	Any IP in subnet	Local network traffic.
`eth.src == 00:11:22:33:44:55`	From specific MAC	Check your MAC; filter device traffic.
`ip`	All IPv4	Excludes IPv6; test on dual-stack.
`ipv6`	All IPv6	Ping ipv6.google.com.

C. Port and Transport Filters

Target TCP/UDP layers.

Filter	What It Does	How to Test
`tcp.port == 443`	TCP on 443 (HTTPS)	Browse secure site.
`udp.port == 53`	UDP on 53 (DNS)	Domain lookup.
`tcp.srcport == 1234`	From source port 1234	Rare; use for custom apps.
`tcp.dstport in {80, 443, 8080}`	To common web ports	Mixed browsing.

D. TCP Flags and Connection Filters

Crucial for troubleshooting.

Filter	What It Does	How to Test
`tcp.flags.syn == 1`	SYN packets (connection starts)	Load page; see handshakes.
`tcp.flags.ack == 1 and tcp.flags.syn == 1`	SYN-ACK	Server responses in handshakes.
`tcp.flags.reset == 1`	RST (resets/connections refused)	Try failed connection.
`tcp.analysis.retransmission`	Retransmitted packets	On slow network; spot duplicates.
`tcp.flags.fin == 1`	FIN (connection ends)	Close browser tab.

E. HTTP-Specific Filters

Deep protocol inspection.

Filter	What It Does	How to Test
`http.request`	HTTP requests (GET/POST)	Browse; see client sends.
`http.request.method == "POST"`	Only POST requests	Submit form.
`http.response.code == 404`	404 errors	Hit bad URL.
`http contains "password"`	Packets with "password"	Login to site.
`http.user_agent contains "Mozilla"`	Browser user agents	Check your browser traffic.

F. String and Regex Filters

Search content.

Filter	What It Does	How to Test
`frame contains "error"`	Any packet with "error"	Capture failed requests.
`dns.qry.name matches ".*google.*"`	DNS for google domains (regex)	Lookup google.com.
`http.host == "example.com"`	Specific host	Visit that site.

G. Time and Length Filters

Performance analysis.

Filter	What It Does	How to Test
`frame.time_relative > 10`	Packets after 10s from first	Long captures.
`http.time > 1`	HTTP responses >1s (slow)	Slow site load.
`frame.len < 100`	Small packets (<100 bytes)	Control packets.

H. Complex Compound Filters

Combine for precision.

- `http and (ip.src == 192.168.1.1 or ip.dst == 192.168.1.1)` : HTTP to/from router.
- `not (dns or arp)` : Everything except DNS/ARP.
- `(tcp.flags.syn == 1 or tcp.flags.fin == 1) and tcp.port == 80` : HTTP connection starts/ends.
- `http.request.uri matches "/login.*" and http contains "user="` : Login attempts.

1. `ip.src == 10.1.11.0/24`

- **Translation:** "Show me all packets **originating** from any device on this specific network segment."
- **The Breakdown:** The `/24` is CIDR notation. Instead of looking for one single IP, this tells Wireshark to look for any IP that starts with `10.1.11.x`.
- **Use Case:** Useful if you want to monitor an entire department or a specific VLAN rather than just one computer.

---

2. `ip.addr == 192.168.1.10 && ip.addr == 192.168.1.20`

- **Translation:** "Show me the **conversation** between these two specific computers."
- **The Breakdown:** The `&&` (AND) operator is the key here. For a packet to show up, it must involve *both* IPs. Usually, one is the source and the other is the destination.
- **Use Case:** Isolating a private chat, a file transfer, or a connection issue between a specific client and a server.

---

3. `tcp.port == 80 || tcp.port == 3389`

- **Translation:** "Show me all web traffic (HTTP) **OR** Remote Desktop (RDP) traffic."
- **The Breakdown:** The `||` (OR) operator broadens your search. If a packet is either port 80 or port 3389, it stays; everything else is hidden.
- **Use Case:** Monitoring a server that acts as both a web server and a remote work terminal.

---

4. `!(ip.addr == 192.168.1.10 && ip.addr == 192.168.1.20)`

- **Translation:** "Show me **everything EXCEPT** the traffic between these two computers."
- **The Breakdown:** The `!` (NOT) symbol negates everything inside the parentheses.
- **Use Case:** If two machines are "spamming" the logs with thousands of packets (like a backup running), use this to hide that noise so you can see what else is happening on the network.

---

5. `(ip.addr == 192.168.1.10 && ip.addr == 192.168.1.20) && (tcp.port == 445 || tcp.port == 139)`

- **Translation:** "Show me **File Sharing (SMB)** traffic between these two specific computers."
- **The Breakdown:** This uses "nested logic."
    - First part: Must be between these two IPs.
    - Second part: Must be using ports 445 or 139 (the ports used for Windows File Sharing/SMB).
- **Use Case:** Troubleshooting why a shared folder isn't opening or why a file transfer is slow.

---

6. `(ip.addr == 192.168.1.10 && ip.addr == 192.168.1.20) && (udp.port == 67 || udp.port == 68)`

- **Translation:** "Show me **DHCP (IP Assignment)** traffic between these two devices."
- **The Breakdown:** * Ports **67** and **68** are used for DHCP.
    - Note that it uses `udp.port` because DHCP doesn't use TCP.
- **Use Case:** Checking if a computer is successfully asking for and receiving an IP address from a router.

---

7. `tcp.dstport == 80`

- **Translation:** "Show me all packets where the **destination** is a web server."
- **The Breakdown:** Unlike `tcp.port`, which looks at both sides, `dstport` specifically looks at where the packet is *going*.
- **Use Case:** Tracking "Outbound" web requests. If you want to see what websites a user is trying to visit, this is the filter to use.

• snmp || dns || icmp–Display the SNMP or DNS or ICMP traffics.

• tcp.port == 25 –Display packets with TCP source or destination port 25.

• tcp.flags–Display packets having a TCP flag.

• tcp.flags == 0x02–Display packets with a TCP SYN flag

PORTS

1. The "Daily Web" Ports

Most of your traffic will fall into these two categories:

- **Port 80 (HTTP):** Standard, unencrypted web traffic. In Wireshark, you can see everything in plain text.
    - *Use Case:* Browsing older sites or testing local web servers.
- **Port 443 (HTTPS):** Encrypted web traffic. In Wireshark, this looks like gibberish unless you have decryption keys.
    - *Use Case:* Almost every modern website (Google, Facebook, YouTube).

---

2. The "Remote Control" Ports

These allow you to access a computer from across the world:

- **Port 22 (SSH):** Secure Shell. Used to log into servers (like Linux) securely.
    - *Use Case:* Managing a website's back-end or a cloud server.
- **Port 23 (Telnet):** The old, unencrypted version of SSH.
    - *Use Case:* **Warning!** Avoid this. If you see Telnet in Wireshark, you can see usernames and passwords in clear text.
- **Port 3389 (RDP):** Remote Desktop Protocol. Used to log into Windows computers visually.
    - *Use Case:* Helping a coworker fix their PC remotely.

---

3. The "Network Foundation" Ports

These work behind the scenes to make the internet function:

- **Port 53 (DNS):** Domain Name System. Translates `google.com` to an IP like `8.8.8.8`.
    - *Use Case:* If you can't load websites but can "ping" an IP, check Port 53 for DNS errors.
- **Ports 67 & 68 (DHCP):** Dynamic Host Configuration Protocol. This is how your phone/PC gets an IP address when it joins WiFi.
    - *Use Case:* Troubleshooting "Limited Connectivity" or IP address conflicts.

---

4. The "File & Mail" Ports

- **Ports 20 & 21 (FTP):** File Transfer Protocol. 21 is for commands (login), 20 is for the actual data.
- **Port 445 (SMB):** Used for Windows file sharing (sharing a folder on your LAN).
- **Port 25 (SMTP):** Used for sending emails between servers.

In a busy network, thousands of packets are flying around simultaneously. Filtering a **Single TCP Stream** allows you to "isolate the conversation." It's like being in a crowded party and suddenly having noise-canceling headphones that only let you hear the person standing right in front of you.

Every TCP conversation is assigned a unique **Stream Index** number by Wireshark (starting at 0).

---

1. How to "Filter In" (Follow) a Stream

If you find a single interesting packet (like a website request or a file download) and want to see the whole story:

1. **Right-click** the packet in the Packet List (top pane).
2. Select **Follow** > **TCP Stream**.

What happens next:

- **A New Window Pops Up:** Wireshark reconstructs the data. Red text is usually data from the *Client* (you), and Blue text is data from the *Server*.
- **The Background Filter Changes:** Wireshark automatically applies a display filter like tcp.stream eq 5.
- **The Result:** Your main window now only shows the handshake, the data exchange, and the closure of that specific connection.

---

2. Using the tcp.stream Filter Manually

You don't have to right-click. You can type these directly into the filter bar:

- **tcp.stream == 0**: Shows the very first TCP conversation captured in the file.
- **tcp.stream == 15**: Shows the 16th conversation.

- **tcp.stream <= 5**: Shows the first 6 conversations (0 through 5).

---

3. "Filtering Out" a Stream

Suppose you are trying to analyze a network issue, but a giant, healthy file download is cluttering your screen. You want to hide that specific conversation to see the "hidden" packets behind it.

The Easy Way:

1. Right-click a packet from the annoying stream.
2. Select **Apply as Filter** > **...and not Selected**.
3. Wireshark will turn the filter bar into: !(tcp.stream == 2).

The Manual Way:

If you know Stream 5 is just "noise," type:

- **!(tcp.stream == 5)**
- **Meaning:** "Show me everything in this capture EXCEPT conversation number 5."

---

4. Real-World Example: Troubleshooting a Slow Login

Imagine you see a login attempt that hangs.

1. Find the packet where you sent your username.
2. Right-click and **Follow > TCP Stream**.
3. Look at the **Time** column in the main window.
4. If you see a 5-second gap between your "Request" and the server's "Response" within that specific stream, you've found exactly where the delay is happening.

---

5. Helpful Attributes to Combine

You can combine stream filtering with the attributes we discussed earlier to be a "Super-Analyst":

Filter	Logic

tcp.stream == 3 && tcp.flags.reset == 1	"Did conversation #3 get cut off by a Reset flag?"
tcp.stream == 10 && frame.len > 1000	"Show me only the big data chunks inside conversation #10."
tcp.stream == 1 && ip.addr == 8.8.8.8	"Confirm if conversation #1 is actually talking to Google."